



École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes
Rabat

Rapport de projet Compilation

Pseudo-code vers un langage de programmation

Réalisé par :

Hiba Rezguani / Ikram Bahbah / Aya Mlouki

Encadré par :

Prof Youness TABII

Année universitaire : 2025–2026

Table des matières

Introduction générale **4**

1 Objectif **5**
Introduction . . . 5
1.1 Objectif principal . . . 5
1.2 Objectifs spécifiques . . . 5
1.3 Portée du projet . . . 5
Conclusion . . . 5

2 Langage (dit alphabet) **6**
Introduction . . . 6
2.1 Mots-clés . . . 6
2.2 Opérateurs . . . 7
2.3 Séparateurs . . . 7
2.4 Identifiants et constantes . . . 7
2.5 Caractères autorisés . . . 8
2.6 Règles de formation . . . 8
Conclusion . . . 8

3 Règles grammaticales **9**
Introduction . . . 9
3.1 Structure du programme . . . 9
3.2 Déclarations . . . 9
3.3 Types de données . . . 9
3.4 Instructions . . . 9
3.5 Affectation . . . 9
3.6 Structures de contrôle . . . 9
3.7 Entrée/Sortie . . . 10
3.8 Expressions . . . 10
3.9 Conditions . . . 10
3.10 Éléments lexicaux . . . 10
Conclusion . . . 10

4 Implémentation du compilateur **11**
Introduction . . . 11
4.1 Analyseur lexical . . . 11
Rôle . . . 11

Structures utilisées . . . 11
Fonctions principales . . . 12
4.2 Analyseur syntaxique . . . 12
Rôle . . . 12
Structures et énumérations utilisées . . . 12
Fonctions de contrôle syntaxique . . . 14
Fonctions correspondant aux règles grammaticales . . . 14
4.3 Analyseur sémantique . . . 14
Rôle . . . 14
Structures utilisées . . . 15
Ajout d'un identificateur . . . 16
Recherche d'un identificateur . . . 17
Vérification de déclaration . . . 17
Compatibilité des types . . . 18
Gestion des erreurs sémantiques . . . 19
Conclusion . . . 19

5 Génération de code **20**
Introduction . . . 20
5.1 Architecture de la génération . . . 20
Approche à deux phases . . . 20
5.2 Fonctions utilitaires de génération . . . 20
Émission de code . . . 20
Conversion des types . . . 21
5.3 Génération des déclarations . . . 21
En-tête du fichier C . . . 21
Génération des structures . . . 22
5.4 Génération des instructions . . . 23
Programme principal . . . 23
Affectation . . . 23
Structure conditionnelle SI . . . 24
Boucle POUR . . . 25
5.5 Génération des entrées/sorties . . . 27
Instruction ECRIRE . . . 27
Instruction LIRE . . . 28
5.6 Génération des expressions . . . 29
Expressions arithmétiques . . . 29
Conditions . . . 30

5.7 Exemple de code généré 30
Pseudo-code source 30
Code C généré 30
Conclusion 31

Conclusion générale **32**

Introduction générale

La compilation représente une étape essentielle dans le développement des logiciels, permettant de transformer un programme écrit dans un langage compréhensible par l'humain en un programme exécutable par la machine. Dans ce contexte, ce projet consiste à concevoir et implémenter un compilateur pour un langage de pseudo-code algorithmique en français, afin de faciliter l'apprentissage de la logique algorithmique.

Le travail débute par la définition formelle du langage, notamment son alphabet et ses règles grammaticales, avant de mettre en œuvre les différentes phases du processus de compilation : l'analyse lexicale, syntaxique et sémantique, suivies de la génération de code.

L'objectif principal est ainsi d'illustrer de manière concrète les concepts fondamentaux de la compilation tout en développant un outil pédagogique fonctionnel.

1 Objectif

Introduction

Ce chapitre présente les objectifs généraux du projet de compilation de pseudo-code.

1.1 Objectif principal

Concevoir et implémenter un compilateur pour un langage de pseudo-code algorithmique en français permettant la traduction vers un langage de programmation cible.

1.2 Objectifs spécifiques

— Définir un langage de pseudo-code structuré et non ambigu

— Établir des règles grammaticales formelles

— Implémenter un analyseur lexical

— Implémenter un analyseur syntaxique

— Implémenter un analyseur sémantique

— Générer du code dans un langage cible

— Fournir un outil pédagogique pour l'apprentissage

1.3 Portée du projet

Le compilateur doit supporter les structures algorithmiques de base :

— Déclarations de variables et constantes

— Structures de contrôle (conditions, boucles)

— Types de données simples et composites

— Opérations arithmétiques et logiques

— Instructions d'entrée/sortie

Conclusion

Ce chapitre a défini les objectifs du projet de compilation.

2 Langage (dit alphabet)

Introduction

Ce chapitre définit l'alphabet du langage pseudo-code.

2.1 Mots-clés

Les mots-clés du langage sont représentés par des tokens numériques :

```
ALGORITHME_TOKEN = 1
DEBUT_TOKEN
FIN_TOKEN
LIRE_TOKEN
ECRIRE_TOKEN
VARIABLES_TOKEN
CONSTANTE_TOKEN
STRUCTURE_TOKEN
FIN_STRUCTURE_TOKEN
SI_TOKEN
ALORS_TOKEN
SINON_TOKEN
FIN_SI_TOKEN
SELON_TOKEN
CAS_TOKEN
DEUX_POINTS_TOKEN
SORTIR_TOKEN
AUTRE_CAS_TOKEN
FIN_SELON_TOKEN
POUR_TOKEN
JUSQUA_TOKEN
PAS_TOKEN
FIN_POUR_TOKEN
TANT_QUE_TOKEN
FIN_TANT_QUE_TOKEN
REPETER_TOKEN
JUSQUA_QUE_TOKEN
ENTIER_TOKEN
REEL_TOKEN
```

```
CARACTERE_TOKEN
CHAINE_TOKEN
BOOLEEN_TOKEN
TABLEAU_TOKEN
DE_TYPE_TOKEN
```

2.2 Opérateurs

```
AFFECTATION_TOKEN      /* := */
PLUS_TOKEN             /* + */
MOINS_TOKEN            /* - */
MULT_TOKEN             /* * */
DIV_TOKEN              /* / */
MOD_TOKEN              /* % */
EGAL_TOKEN             /* = */
DIFFERENT_TOKEN        /* <> */
INFERIEUR_TOKEN        /* < */
INFERIEUR_EGAL_TOKEN   /* <= */
SUPERIEUR_TOKEN        /* > */
SUPERIEUR_EGAL_TOKEN   /* >= */
ET_TOKEN
OU_TOKEN
NON_TOKEN
```

2.3 Séparateurs

```
PARENTHESE_OUVRANTE_TOKEN  /* ( */
PARENTHESE_FERMANTE_TOKEN  /* ) */
CROCHET_OUVRANT_TOKEN      /* [ */
CROCHET_FERMANT_TOKEN      /* ] */
VIRGULE_TOKEN              /* , */
POINT_VIRGULE_TOKEN        /* ; */
POINT_TOKEN                /* . */
```

2.4 Identifiants et constantes

```
ID_TOKEN                /* Identificateur */
NUM_ENTIER_TOKEN        /* Nombre entier */
NUM_REEL_TOKEN          /* Nombre réel */
```

```
CHAINE_LITTERALE_TOKEN   /* Chaîne de caractères */
FIN_FICHIER_TOKEN        /* Fin de fichier */
```

2.5 Caractères autorisés

— Lettres (a-z, A-Z)

— Chiffres (0-9)

— Caractères spéciaux : + - * /

— Espaces, tabulations, retours à la ligne

2.6 Règles de formation

— Identificateurs : lettre{lettre|chiffre|_}

— Nombres entiers : chiffre{chiffre}

— Nombres réels : chiffre{chiffre}.chiffre{chiffre}

— Chaînes : "{caractère}"

Conclusion

Ce chapitre a défini l'alphabet complet du langage pseudo-code.

3 Règles grammaticales

Introduction

Ce chapitre présente les règles grammaticales du langage en BNF.

3.1 Structure du programme

```
PROGRAM ::= algorithme ID ; BLOCK
BLOCK ::= VARIABLES CONSTANTES INSTS_PRINCIPAL
INSTS_PRINCIPAL ::= debut INST { ; INST } fin
```

3.2 Déclarations

```
VARIABLES ::= variables LISTE_VARS | ε
LISTE_VARS ::= ID { , ID } : TYPE ; { ID { , ID } : TYPE ; }
CONSTANTES ::= constante ID = NUM ; { ID = NUM ; } | ε
```

3.3 Types de données

```
TYPE ::= TYPE_BASE | tableau [ NUM ] de TYPE_BASE | structure CHAMPS fin_structure
TYPE_BASE ::= entier | reel | caractere | chaine | booleen | ID
CHAMPS ::= ID { , ID } : TYPE_BASE { ; ID { , ID } : TYPE_BASE }
```

3.4 Instructions

```
INST ::= AFFEC | SI | SELON | POUR | TANT_QUE | REPETER | ECRIRE | LIRE | ε
INSTS_CORPS ::= INST { ; INST} | ε
```

3.5 Affectation

```
VARIABLE ::= ID [ . ID ] [ [ EXPR ] ]
AFFEC ::= VARIABLE := ( EXPR | COND )
```

3.6 Structures de contrôle

```
SI ::= si COND alors INSTS_CORPS [ sinon INSTS_CORPS | ε ] fin_si
SELON ::= selon ( ID ) LISTE_CAS [ CAS_AUTRES ] fin_selon
LISTE_CAS ::= si VALEUR : INSTS_CAS , sortir { si VALEUR : INSTS_CAS , sortir }
CAS_AUTRES ::= si autres : INSTS_CAS | ε
```

```
INSTS_CAS ::= INST { ; INST }
POUR ::= pour ID := EXPR jusqua EXPR [ pas EXPR ] faire INSTS_CORPS fin_pour
TANT_QUE ::= tant_que COND faire INSTS_CORPS fin_tant_que
REPETER ::= repeter INSTS_CORPS jusqua_que COND
```

3.7 Entrée/Sortie

```
ECRIRE ::= ecrire ( EXPR { , EXPR } )
LIRE ::= lire ( VARIABLE { , VARIABLE } )
```

3.8 Expressions

```
EXPR ::= TERM { ADDOP TERM }
ADDOP ::= + | - | ou
TERM ::= FACT { MULOP FACT }
MULOP ::= * | / | et | mod
FACT ::= CHAINE | non FACT | NUM_FACT
NUM_FACT ::= VARIABLE | NUM | ( EXPR ) | SIGNE NUM_FACT
SIGNE ::= + | -
```

3.9 Conditions

```
COND ::= COND_TERM { ou COND_TERM }
COND_TERM ::= COND_FACT { et COND_FACT }
COND_FACT ::= non COND_FACT | ( COND ) | EXPR [ RELOP EXPR ]
RELOP ::= = | <> | < | > | <= | >=
```

3.10 Éléments lexicaux

```
ID ::= lettre { lettre | chiffre | _ }
NUM ::= chiffre { chiffre } [ . chiffre { chiffre } ]
CHAINE ::= " { caractère } "
VALEUR ::= NUM | CHAINE
```

Conclusion

Ce chapitre a présenté les règles grammaticales complètes du langage.

4 Implémentation du compilateur

Introduction

Ce chapitre décrit l'implémentation du compilateur du langage de pseudo-code algorithmique. Il présente l'architecture générale adoptée ainsi que les différents analyseurs mis en œuvre : l'analyseur lexical, l'analyseur syntaxique et l'analyseur sémantique.

4.1 Analyseur lexical

Rôle

L'analyseur lexical est chargé de lire le fichier source caractère par caractère et de le transformer en une suite de symboles lexicaux appelés *tokens*. Ces tokens sont ensuite transmis à l'analyseur syntaxique. Il ignore les séparateurs tels que les espaces, les tabulations et les commentaires, et détecte les erreurs lexicales dès leur apparition.

Structures utilisées

L'analyseur lexical s'appuie sur plusieurs structures de données :

— **CODE_LEX** : énumération représentant l'ensemble des tokens du langage (mots-clés, opérateurs, séparateurs, identificateurs, constantes et fin de fichier).

— **TSym_Cour** : structure représentant le symbole courant analysé, contenant le code du token, son nom et éventuellement sa valeur numérique.

```
typedef struct {
    CODE_LEX CODE;
    char NOM[MAX_ID + 1];
    double val;
} TSym_Cour;
```

— **MotCle** : structure associant chaque mot-clé du langage à son token correspondant.

```
typedef struct {
    char nom[20];
    CODE_LEX code;
} MotCle;
```

— **Erreurs_Lexicaux** : énumération des différents types d'erreurs lexicales possibles.

```
typedef enum {
    ERR_CAR_INC,      // caractere inconnu
    ERR_FICH_VID,     // fichier vide
```

```
    ERR_ID_LONG,        // identificateur trop long
    ERR_NUM_LONG,       // constante numerique trop longue
    ERR_CHAINE_INC      // chaine non fermee
} Erreurs_Lexicaux;
```

— **Message_Erreur_Lexical** : table associant chaque erreur lexicale à un message explicite.

Fonctions principales

Les principales fonctions de l'analyseur lexical sont :

— `Lire_Car()` : lit le caractère suivant du fichier source.

— `Passer_Separateurs()` : ignore les espaces, retours à la ligne et commentaires.

— `Lire_Mot()` : reconnaît les identificateurs et les mots-clés.

— `Lire_Nombre()` : analyse les constantes numériques.

— `Lire_Chaine()` : analyse les chaînes de caractères.

— `Lire_Special()` : reconnaît les opérateurs et symboles spéciaux.

— `Rechercher_Mot_Cle()` : vérifie si un identificateur est un mot-clé.

— `Sym_Suiv()` : retourne le prochain token à l'analyseur syntaxique.

— `Erreur_Lexicale()` : signale une erreur lexicale et arrête la compilation.

4.2 Analyseur syntaxique

Rôle

L'analyseur syntaxique vérifie que la suite de tokens produite par l'analyseur lexical respecte la grammaire du langage. Il est implémenté à l'aide d'une méthode de descente récursive, où chaque fonction correspond à une règle grammaticale.

Structures et énumérations utilisées

L'analyseur syntaxique utilise les structures suivantes :

— **Erreur_Syntaxique_Code** : énumération des différents types d'erreurs syntaxiques.

```
typedef enum {
    SYN_ERR_ALGORITHME,
    SYN_ERR_ID,
    SYN_ERR_DEBUT,
    SYN_ERR_FIN,
```

```
6       SYN_ERR_POINT_VIRGULE,
7       SYN_ERR_PAR_FERM,
8       SYN_ERR_CROCHET_FERM,
9       SYN_ERR_CROCHET_OUV,
10      SYN_ERR_DEUX_POINTS,
11      SYN_ERR_VIRGULE,
12      SYN_ERR_AFFECTATION,
13      SYN_ERR_EGAL,
14      SYN_ERR_CONSTANTE,
15      SYN_ERR_VARIABLES,
16      SYN_ERR_TYPE,
17      SYN_ERR_TABLEAU,
18      SYN_ERR_STRUCTURE,
19      SYN_ERR_FIN_STRUCTURE,
20      SYN_ERR_SI,
21      SYN_ERR_ALORS,
22      SYN_ERR_SELON,
23      SYN_ERR_FIN_SELON,
24      SYN_ERR_POUR,
25      SYN_ERR_FIN_POUR,
26      SYN_ERR_TANT_QUE,
27      SYN_ERR_FIN_TANT_QUE,
28      SYN_ERR_REPETER,
29      SYN_ERR_JUSQUA_QUE,
30      SYN_ERR_ECRIRE,
31      SYN_ERR_LIRE,
32      SYN_ERR_RELOP,
33      SYN_ERR_SORTIR,
34      SYN_ERR_AUTRE_CAS,
35      SYN_ERR_VALEUR,
36      SYN_ERR_SIGNE,
37      SYN_ERR_FACT,
38      SYN_ERR_NUM,
39      SYN_ERR_NON_TERM,
40      SYN_ERR_DE_TYPE
41  } Erreur_Syntaxique_Code;
```

— **Message_Erreur_Syntaxique** : table associant chaque erreur syntaxique à un message explicite.

```
1  typedef struct {
```

```
2      Erreur_Syntaxique_Code CODE_ERR;
3      char mes[40];
4  } Message_Erreur_Syntaxique;
```

Fonctions de contrôle syntaxique

Les fonctions suivantes assurent le contrôle du flux syntaxique :

— `Test_Symbole()` : vérifie que le symbole courant correspond à celui attendu.

```
1  void Test_Symbole(CODE_LEX attendu, Erreur_Syntaxique_Code err
       ) {
2      if (Sym_Cour.CODE == attendu) {
3          Sym_Suiv();
4      } else {
5          Erreur_Syntaxique(err);
6      }
7  }
```

— `Erreur_Syntaxique()` : affiche un message d'erreur syntaxique et arrête l'analyse.

Fonctions correspondant aux règles grammaticales

Chaque règle de la grammaire est implémentée par une fonction dédiée, notamment :

— `PROGRAM()`, `BLOCK()`

— `VARIABLES()`, `LISTE_VARS()`, `CONSTANTES()`

— `INSTS_PRINCIPAL()`, `INSTS_CORPS()`, `INST()`

— `AFFEC()`, `SI()`, `SELON()`

— `POUR()`, `TANT_QUE()`, `REPETER()`

— `ECRIRE()`, `LIRE()`

— `EXPR()`, `TERM()`, `FACT()`

Ces fonctions assurent la reconnaissance structurelle du programme sans encore vérifier la cohérence sémantique.

4.3 Analyseur sémantique

Rôle

L'analyseur sémantique vérifie la cohérence logique du programme analysé. Il s'assure notamment que les identificateurs sont déclarés, que les types sont compatibles et que les règles sémantiques du langage sont respectées.

Cette analyse est intégrée directement aux fonctions de l'analyseur syntaxique.

Structures utilisées

Les structures suivantes sont utilisées pour la gestion sémantique :

— **TSYM** : type de symbole (programme, variable, constante, etc.).

```
typedef enum {
    TPROG,      // Programme
    TCONST,     // Constante
    TVAR,       // Variable
    TSTRUCT,    // Champ de structure
    TTYPE       // Type defini
} TSYM;
```

— **TYPE_DONNEES** : type de données du langage.

```
typedef enum {
    TYPE_ENTIER,
    TYPE_REEL,
    TYPE_CARACTERE,
    TYPE_CHAINE,
    TYPE_BOOLEEN,
    TYPE_TABLEAU,
    TYPE_STRUCT,
    TYPE_UNDEFINED
} TYPE_DONNEES;
```

— **T_TAB_IDF** : structure représentant une entrée de la table des symboles. Elle contient les informations sémantiques associées à chaque identificateur (nom, nature, type, valeur, adresse mémoire, taille, etc.).

```
typedef struct {
    char NOM[20];
    TSYM TIDF;
    TYPE_DONNEES TYPE;
    int VAL; // Valeur pour les constantes
    int ADRESSE; // Adresse memoire pour les variables
    int TAILLE; // Taille pour les tableaux ou structures
    int OFFSET;  // Deplacement pour les champs de structure
    struct {
        char NOM[20];
        TYPE_DONNEES TYPE;
```

```
12      } CHAMPS[20]; // Champs d'une structure
13      int NB_CHAMPS;
14  } T_TAB_IDF;
```

— **Table des symboles** : tableau global contenant l'ensemble des identificateurs déclarés dans le programme, utilisé pour la vérification des règles sémantiques.

```
1  T_TAB_IDF TAB_SYM[TAILLE_TABLE_SYM];
2  int NBR_SYM = 0;
```

— **Erreur_Semantique_Code** : énumération des erreurs détectées lors de l'analyse sémantique. Ces erreurs concernent le sens du programme, notamment la déclaration des identificateurs, la compatibilité des types, l'utilisation des constantes et l'accès aux structures et tableaux.

```
1  typedef enum {
2      SEM_ERR_DOUBLE_DECL,  // Double declaration
3      SEM_ERR_ID_NON_DECL,  // Identificateur non declare
4      SEM_ERR_CONST_MODIF,  // Modification d'une constante
5      SEM_ERR_TYPE_MISMATCH,  // Incompatibilite de type
6      SEM_ERR_PROG_ID_USE,   // Utilisation du nom du programme
7      SEM_ERR_STRUCT_CHAMP,   // Champ de structure inexistant
8      SEM_ERR_INDEX_TAB,     // Indice de tableau invalide
9      SEM_ERR_ID_TROP_LONG   // Identificateur trop long
10 } Erreur_Semantique_Code;
```

Ajout d'un identificateur

La fonction suivante permet d'ajouter un nouvel identificateur dans la table des symboles après vérification de l'absence de double déclaration.

```
1  int Ajouter_Sym(char *nom, TSYM tidf, TYPE_DONNEES type, int val,
       int adresse) {
2      if (NBR_SYM >= TAILLE_TABLE_SYM) {
3  Erreur_Semantique_Format(SEM_ERR_INDEX_TAB, "table␣des␣symboles␣
       pleine");
4          return -1;
5      }
6      if (tidf != TTYPE) {
7          if (Rechercher_Sym(nom) != -1) {  Erreur_Semantique_Format(
               SEM_ERR_DOUBLE_DECL, "%s", nom);
8              return -1;
```

```
9           }
10      }
11      if (strlen(nom) >= MAX_ID) { Erreur_Semantique_Format(
            SEM_ERR_ID_TROP_LONG, "%s (max %d caracteres)", nom, MAX_ID)
            ;
12      }
13      strcpy(TAB_SYM[NBR_SYM].NOM, nom);
14      TAB_SYM[NBR_SYM].TIDF = tidf;
15      TAB_SYM[NBR_SYM].TYPE = type;
16      TAB_SYM[NBR_SYM].VAL = val;
17      TAB_SYM[NBR_SYM].ADRESSE = adresse;
18      TAB_SYM[NBR_SYM].TAILLE = 0; TAB_SYM[NBR_SYM].NB_CHAMPS = 0;
19      if (tidf == TVAR) {
20          if (type == TYPE_STRUCT || type == TYPE_TABLEAU) {
               TAB_SYM[NBR_SYM].ADRESSE = OFFSET;
21          } else {
              OFFSET++;
22          TAB_SYM[NBR_SYM].ADRESSE = OFFSET;
23          }
24      }
25      return NBR_SYM++;
26  }
27
```

Recherche d'un identificateur

Cette fonction permet de vérifier l'existence d'un identificateur dans la table des symboles avant son utilisation.

```
1  int Rechercher_Sym(char *nom) {
2      for (int i = 0; i < Indice_Sym; i++) {
3          if (strcmp(TAB_SYM[i].nom, nom) == 0)
4              return i;
5      }
6      return -1;
7  }
```

Vérification de déclaration

La fonction suivante vérifie qu'un identificateur est déclaré avant toute utilisation.

```
1  int Verifier_ID_Declare(char *nom, int avant_begin) {
```

```
2     int index = Rechercher_Sym(nom);
3     if (index == -1) {
4         if (avant_begin) {
5             return -1;
6         } else {
7             Erreur_Semantique_Format(SEM_ERR_ID_NON_DECL, "%s", nom
                  );
8             return -1;
9         }
10    }
11    INDEX_DER_SYM_ACC = index;
12    return index;
13 }
```

Compatibilité des types

Cette fonction permet de vérifier la compatibilité des types lors des affectations et des expressions.

```
1  int Types_Compatibles(TYPE_DONNEES type_dest, TYPE_DONNEES type_src
       , int pour_affectation) {
2     if (type_dest == type_src) return 1;
3     // Pour les affectations : regles strictes
4     if (pour_affectation) {
5         if (type_dest == TYPE_ENTIER && type_src == TYPE_BOOLEEN)
              return 1;
6         if (type_dest == TYPE_REEL && type_src == TYPE_ENTIER)
              return 1;
7         if (type_dest == TYPE_BOOLEEN && type_src == TYPE_ENTIER)
              return 1;
8         return 0;
9     }
10    // Pour les operations : regles plus permissives
11    if (type_dest == TYPE_ENTIER && type_src == TYPE_REEL)
12    return 1;
13    if (type_dest == TYPE_REEL && type_src == TYPE_ENTIER)
14    return 1;
15    if (type_dest == TYPE_ENTIER && type_src == TYPE_BOOLEEN)
          return 1;
16    if (type_dest == TYPE_BOOLEEN && type_src == TYPE_ENTIER)
          return 1;
```

```
17      // Types UNDEFINED acceptes partout (pour les declarations)
18      if (type_dest == TYPE_UNDEFINED || type_src == TYPE_UNDEFINED)
            return 1;
19      return 0;
20  }
```

Gestion des erreurs sémantiques

Les erreurs sémantiques sont signalées par une fonction dédiée qui affiche un message explicite et interrompt la compilation.

```
1  void Erreur_Semantique(char *message) {
2      printf("Erreur␣semantique:␣%s\n", message);
3      exit(EXIT_FAILURE);
4  }
```

Conclusion

Ce chapitre a présenté l'implémentation du compilateur à travers ses trois composantes principales : l'analyse lexicale, l'analyse syntaxique et l'analyse sémantique. Cette organisation permet une détection progressive et précise des erreurs à chaque étape du processus de compilation.

5 Génération de code

Introduction

Ce chapitre présente la phase finale du compilateur : la génération de code C à partir du pseudo-code analysé. Cette phase transforme la représentation interne du programme en code C exécutable, en exploitant les informations collectées lors des analyses précédentes.

5.1 Architecture de la génération

Approche à deux phases

Le compilateur utilise une approche à deux passes pour la génération de code :

— **Phase 1 (MODE_GENERATION = 0)** : Analyse complète du programme pour construire la table des symboles et vérifier la sémantique.

— **Phase 2 (MODE_GENERATION = 1)** : Génération effective du code C en parcourant à nouveau le programme source.

Cette approche garantit que toutes les informations nécessaires (types, déclarations, structures) sont disponibles avant de générer le code.

5.2 Fonctions utilitaires de génération

Émission de code

Trois fonctions principales permettent d'émettre du code avec gestion de l'indentation :

```
1  void Emettre(char *format, ...) {
2      va_list args;
3      va_start(args, format);
4      vfprintf(Fichier_C, format, args);
5      va_end(args);
6  }
7
8  void EmettreIndente(char *format, ...) {
9      for (int i = 0; i < NIVEAU_INDENTATION; i++) {
10         fprintf(Fichier_C, "␣␣␣␣");
11     }
12     va_list args;
13     va_start(args, format);
14     vfprintf(Fichier_C, format, args);
15     va_end(args);
```

```
16 }
17
18 void EmettreLigne(char *format, ...) {
19     for (int i = 0; i < NIVEAU_INDENTATION; i++) {
20         fprintf(Fichier_C, "␣␣␣␣");
21     }
22     va_list args;
23     va_start(args, format);
24     vfprintf(Fichier_C, format, args);
25     va_end(args);
26     fprintf(Fichier_C, "\n");
27 }
```

Conversion des types

La fonction `Type_Vers_C` convertit les types du pseudo-code en types C :

```
1  char* Type_Vers_C(TYPE_DONNEES type) {
2      switch (type) {
3          case TYPE_ENTIER: return "int";
4          case TYPE_REEL: return "double";
5          case TYPE_CARACTERE: return "char";
6          case TYPE_CHAINE: return "char";
7          case TYPE_BOOLEEN: return "bool";
8          default: return "int";
9      }
10 }
```

5.3 Génération des déclarations

En-tête du fichier C

La fonction `Generer_Declarations_Globales` génère l'en-tête complet :

```
1  void Generer_Declarations_Globales() {
2      EmettreLigne("#include␣<stdio.h>");
3      EmettreLigne("#include␣<stdlib.h>");
4      EmettreLigne("#include␣<string.h>");
5      EmettreLigne("#include␣<stdbool.h>");
6      EmettreLigne("");
7
8      // Definir les structures (typedef)
```

```
9       for (int i = 0; i < NBR_SYM; i++) {
10          if (TAB_SYM[i].TIDF == TTYPE &&
11              TAB_SYM[i].TYPE == TYPE_STRUCT) {
12              Generer_Definition_Structure(i);
13          }
14      }
15
16      // Declarer les constantes
17      for (int i = 0; i < NBR_SYM; i++) {
18          if (TAB_SYM[i].TIDF == TCONST) {
19              Generer_Declaration_Constante(
20                  TAB_SYM[i].NOM, TAB_SYM[i].VAL);
21          }
22      }
23
24      // Declarer les variables
25      for (int i = 0; i < NBR_SYM; i++) {
26          if (TAB_SYM[i].TIDF == TVAR) {
27              Generer_Declaration_Variable(
28                  TAB_SYM[i].NOM, TAB_SYM[i].TYPE,
29                  TAB_SYM[i].TYPE_ELEM, TAB_SYM[i].TAILLE);
30          }
31      }
32  }
```

Génération des structures

```
1   void Generer_Definition_Structure(int type_index) {
2       if (TAB_SYM[type_index].TIDF != TTYPE ||
3           TAB_SYM[type_index].TYPE != TYPE_STRUCT) return;
4
5       EmettreLigne("typedef␣struct␣{");
6       NIVEAU_INDENTATION++;
7
8       for (int j = 0; j < TAB_SYM[type_index].NB_CHAMPS; j++) {
9           char *nom_champ = TAB_SYM[type_index].CHAMPS[j].NOM;
10          TYPE_DONNEES t = TAB_SYM[type_index].CHAMPS[j].TYPE;
11
12          if (t == TYPE_CHAINE) {
13              EmettreLigne("char␣%s[%d];", nom_champ, MAX_CHAINE);
```

```
14          } else {
15              EmettreLigne("%s␣%s;", Type_Vers_C(t), nom_champ);
16          }
17      }
18
19      NIVEAU_INDENTATION--;
20      EmettreLigne("}␣%s;", TAB_SYM[type_index].NOM);
21  }
```

5.4 Génération des instructions

Programme principal

Le bloc `debut...fin` est transformé en fonction `main()` :

```
1   void INSTS_PRINCIPAL() {
2       Test_Symbole(DEBUT_TOKEN, SYN_ERR_DEBUT);
3
4       if (MODE_GENERATION) {
5           EmettreLigne("int␣main()␣{");
6           NIVEAU_INDENTATION++;
7       }
8
9       INST();
10      while (Sym_Cour.CODE == POINT_VIRGULE_TOKEN) {
11          Sym_Suiv();
12          INST();
13      }
14
15      Test_Symbole(FIN_TOKEN, SYN_ERR_FIN);
16
17      if (MODE_GENERATION) {
18          EmettreLigne("return␣0;");
19          NIVEAU_INDENTATION--;
20          EmettreLigne("}");
21      }
22  }
```

Affectation

La génération des affectations gère les cas spéciaux des chaînes et des booléens :

```
void AFFEC() {
    // Capturer la variable cible (lvalue)
    CAPTURE_LVALUE = 1;
    VARIABLE();
    CAPTURE_LVALUE = 0;
    type_dest = TYPE_CIBLE_AFFEC;

    Test_Symbole(AFFECTATION_TOKEN, SYN_ERR_AFFECTATION);

    if (MODE_GENERATION) {
        EmettreIndente("");

        if (type_dest == TYPE_CHAINE) {
            // strcpy pour les chaines
            Emettre("strcpy(%s, ", LVALUE_BUF);
            GENERER_EXPR();
            Emettre(");\n");
        } else if (type_dest == TYPE_BOOLEEN) {
            // Condition pour les booleens
            Emettre("%s = ", LVALUE_BUF);
            GENERER_COND();
            Emettre(";\n");
        } else {
            // Affectation simple
            Emettre("%s = ", LVALUE_BUF);
            GENERER_EXPR();
            Emettre(";\n");
        }
    }
}
```

Structure conditionnelle SI

```
void SI() {
    Test_Symbole(SI_TOKEN, SYN_ERR_SI);

    if (MODE_GENERATION) {
        EmettreIndente("if (");
        GENERER_COND();
        Emettre(") {\n");
```

```
8           NIVEAU_INDENTATION++;
9       }
10
11      Test_Symbole(ALORS_TOKEN, SYN_ERR_ALORS);
12      INSTS_CORPS();
13
14      if (MODE_GENERATION) {
15          NIVEAU_INDENTATION--;
16          EmettreIndente("}");
17      }
18
19      if (Sym_Cour.CODE == SINON_TOKEN) {
20          Sym_Suiv();
21          if (MODE_GENERATION) {
22              Emettre("␣else␣{\n");
23              NIVEAU_INDENTATION++;
24          }
25          INSTS_CORPS();
26          if (MODE_GENERATION) {
27              NIVEAU_INDENTATION--;
28              EmettreIndente("}");
29          }
30      }
31
32      if (MODE_GENERATION) Emettre("\n");
33      Test_Symbole(FIN_SI_TOKEN, SYN_ERR_SI);
34  }
```

Boucle POUR

```
1   void POUR() {
2       Test_Symbole(POUR_TOKEN, SYN_ERR_POUR);
3
4       char nom_compteur[20];
5       strcpy(nom_compteur, Sym_Cour.NOM);
6       Test_Symbole(ID_TOKEN, SYN_ERR_ID);
7
8       if (MODE_GENERATION) {
9           EmettreIndente("for␣(%s␣=␣", nom_compteur);
10      }
```

```
11
12      Test_Symbole(AFFECTATION_TOKEN, SYN_ERR_AFFECTATION);
13      GENERER_EXPR();
14
15      if (MODE_GENERATION) {
16          Emettre("; %s <= ", nom_compteur);
17      }
18
19      Test_Symbole(JUSQUA_TOKEN, SYN_ERR_JUSQUA_QUE);
20      GENERER_EXPR();
21
22      if (Sym_Cour.CODE == PAS_TOKEN) {
23          Sym_Suiv();
24          if (MODE_GENERATION) {
25              Emettre("; %s += ", nom_compteur);
26              GENERER_EXPR();
27          }
28      } else {
29          if (MODE_GENERATION) {
30              Emettre("; %s++", nom_compteur);
31          }
32      }
33
34      if (MODE_GENERATION) {
35          Emettre(") {\n");
36          NIVEAU_INDENTATION++;
37      }
38
39      Test_Symbole(FAIRE_TOKEN, SYN_ERR_ALORS);
40      INSTS_CORPS();
41
42      if (MODE_GENERATION) {
43          NIVEAU_INDENTATION--;
44          EmettreIndente("}\n");
45      }
46
47      Test_Symbole(FIN_POUR_TOKEN, SYN_ERR_FIN_POUR);
48  }
```

5.5 Génération des entrées/sorties

Instruction ECRIRE

L'instruction `ecrire` est transformée en `printf` avec construction dynamique du format :

```
void ECRIRE() {
    Test_Symbole(ECRIRE_TOKEN, SYN_ERR_ECRIRE);
    Test_Symbole(PARENTHESE_OUVRANTE_TOKEN, SYN_ERR_PAR_FERM);

    if (MODE_GENERATION) {
        Init_Printf_Buffers();
    }

    // Traiter les expressions a afficher
    GENERER_EXPR_ECRIRE();

    while (Sym_Cour.CODE == VIRGULE_TOKEN) {
        Sym_Suiv();
        GENERER_EXPR_ECRIRE();
    }

    if (MODE_GENERATION) {
        // Emettre le printf final
        EmettreIndente("printf(\"%s\"", FORMAT_PRINTF);
        if (ARGS_INDEX > 0) {
            Emettre(",␣%s", ARGS_PRINTF);
        }
        Emettre(");\n");
    }

    Test_Symbole(PARENTHESE_FERMANTE_TOKEN, SYN_ERR_PAR_FERM);
}
```

La fonction `GENERER_EXPR_ECRIRE` construit le format et les arguments :

```
void GENERER_EXPR_ECRIRE() {
    if (Sym_Cour.CODE == CHAINE_LITTERALE_TOKEN) {
        // Chaine litterale : ajouter au format
        Ajouter_Format_Printf(Sym_Cour.NOM);
        Sym_Suiv();
    } else if (Sym_Cour.CODE == ID_TOKEN) {
```

```
7           // Variable : determiner le type et ajouter format + arg
8           int index = Rechercher_Sym(Sym_Cour.NOM);
9           TYPE_DONNEES type_arg = TAB_SYM[index].TYPE;
10
11          if (type_arg == TYPE_REEL) {
12              Ajouter_Format_Printf("%lf");
13              Ajouter_Argument_Printf(Sym_Cour.NOM);
14          } else if (type_arg == TYPE_CHAINE) {
15              Ajouter_Format_Printf("%s");
16              Ajouter_Argument_Printf(Sym_Cour.NOM);
17          } else if (type_arg == TYPE_BOOLEEN) {
18              Ajouter_Format_Printf("%s");
19              char bool_expr[170];
20              snprintf(bool_expr, sizeof(bool_expr),
21                  "(%s␣?␣\"vrai\"␣:␣\"faux\")", Sym_Cour.NOM);
22              Ajouter_Argument_Printf(bool_expr);
23          } else {
24              Ajouter_Format_Printf("%d");
25              Ajouter_Argument_Printf(Sym_Cour.NOM);
26          }
27          Sym_Suiv();
28      }
29  }
```

Instruction LIRE

L'instruction `lire` est transformée en `scanf` :

```
1   void LIRE() {
2       Test_Symbole(LIRE_TOKEN, SYN_ERR_LIRE);
3       Test_Symbole(PARENTHESE_OUVRANTE_TOKEN, SYN_ERR_PAR_FERM);
4
5       char nom_var[20];
6       strcpy(nom_var, Sym_Cour.NOM);
7
8       TYPE_DONNEES type_var = TAB_SYM[var_index].TYPE;
9
10      if (MODE_GENERATION) {
11          EmettreIndente("scanf(\"");
12          if (type_var == TYPE_ENTIER) {
13              Emettre("%%d");
```

```
14          } else if (type_var == TYPE_REEL) {
15              Emettre("%%lf");
16          } else if (type_var == TYPE_CHAINE) {
17              Emettre("%%s");
18          }
19          Emettre("\", &%s);\n", nom_var);
20      }
21
22      VARIABLE();
23
24      Test_Symbole(PARENTHESE_FERMANTE_TOKEN, SYN_ERR_PAR_FERM);
25  }
```

5.6 Génération des expressions

Expressions arithmétiques

```
1   void GENERER_EXPR() {
2       GENERER_TERM();
3
4       while (Sym_Cour.CODE == PLUS_TOKEN ||
5              Sym_Cour.CODE == MOINS_TOKEN ||
6              Sym_Cour.CODE == OU_TOKEN) {
7           CODE_LEX op = Sym_Cour.CODE;
8           Sym_Suiv();
9
10          if (MODE_GENERATION) {
11              if (op == OU_TOKEN) {
12                  Emettre(" || ");
13              } else if (op == PLUS_TOKEN) {
14                  Emettre(" + ");
15              } else {
16                  Emettre(" - ");
17              }
18          }
19
20          GENERER_TERM();
21      }
22  }
```

Conditions

```
void GENERER_COND() {
    GENERER_COND_TERM();

    while (Sym_Cour.CODE == OU_TOKEN) {
        Sym_Suiv();
        if (MODE_GENERATION) Emettre("␣||␣");
        GENERER_COND_TERM();
    }
}

void GENERER_RELOP(CODE_LEX op) {
    switch (op) {
        case EGAL_TOKEN: Emettre("␣==␣"); break;
        case DIFFERENT_TOKEN: Emettre("␣!=␣"); break;
        case INFERIEUR_TOKEN: Emettre("␣<␣"); break;
        case SUPERIEUR_TOKEN: Emettre("␣>␣"); break;
        case INFERIEUR_EGAL_TOKEN: Emettre("␣<=␣"); break;
        case SUPERIEUR_EGAL_TOKEN: Emettre("␣>=␣"); break;
    }
}
```

5.7 Exemple de code généré

Pseudo-code source

```
algorithme exemple;
variables
    x, y : entier;
    resultat : reel;
debut
    lire(x, y);
    resultat := (x + y) / 2;
    ecrire("La moyenne est : ", resultat)
fin
```

Code C généré

```
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

int x;
int y;
double resultat;

int main() {
    scanf("%d", &x);
    scanf("%d", &y);
    resultat = (x + y) / 2;
    printf("La␣moyenne␣est␣:␣%lf", resultat);
    return 0;
}
```

Conclusion

Ce chapitre a présenté la phase de génération de code du compilateur. L'approche à deux passes permet de garantir la correction sémantique avant la génération, tandis que les fonctions dédiées assurent une émission de code C lisible et correctement indenté. Le système de génération gère efficacement les spécificités du langage cible, notamment pour les chaînes de caractères et les structures de données composites.

Conclusion générale

Ce projet avait pour objectif de concevoir et d'implémenter un compilateur capable de traduire un langage de pseudo-code algorithmique en français vers le langage C. Il nous a permis de mettre en pratique les concepts fondamentaux de la compilation, depuis la définition du langage jusqu'à la génération d'un programme exécutable.

Nous avons d'abord formalisé le langage en définissant son alphabet et ses règles grammaticales, garantissant une structure claire et non ambiguë. Ensuite, nous avons implémenté les différentes phases du compilateur : l'analyse lexicale pour la reconnaissance des tokens, l'analyse syntaxique pour la vérification de la structure du programme, et l'analyse sémantique pour assurer sa cohérence grâce à la gestion de la table des symboles et au contrôle des types.

Enfin, la phase de génération de code a permis de transformer automatiquement le pseudo-code en code C, en produisant les déclarations, les structures de contrôle ainsi que les instructions d'entrée et de sortie. Cette étape représente l'aboutissement du processus de compilation.

Ce projet constitue ainsi une expérience enrichissante qui nous a permis de mieux comprendre l'architecture des compilateurs et de renforcer nos compétences en conception de langages et en développement logiciel.